TECHNICAL REPORT | VERSION 2.0

Rwanda Agri Stability System (RASS)

Architecture, district-level price regulation, and out-of-sample evaluation of market price forecasting on WFP Rwanda retail prices

Authors

Iriza Gatera M. • Atete Muhire G. V. • Gaju S. • Ngabonziza H.

Supervisor

Mr. Laurent Maniraho

Institution

Faculty of Information Technology, Adventist University of Central Africa (AUCA), Kigali, Rwanda

Prepared for

Ministry of Agriculture and Animal Resources (MINAGRI) and the Rwanda Agriculture and Animal Resources Development Board (RAB)

DOI 10.5281/zenodo.23870262

Kigali, November 2026. This report supersedes the *RASS Executive Research Brief* (2026) and the figures shown in earlier demonstrations of the platform.

Document control

Title	Rwanda Agri Stability System (RASS): Technical Report
Version	2.0 (November 2026)
DOI	https://doi.org/10.5281/zenodo.23870262
Licence	Creative Commons Attribution 4.0 International (CC BY 4.0)
Status	Published on Zenodo; submitted to MINAGRI and RAB for review
Authors	Iriza Gatera M., Atete Muhire G. V., Gaju S., Ngabonziza H.
Supervisor	Mr. Laurent Maniraho, Faculty of Information Technology, AUCA
Supersedes	*RASS: Executive Research Brief* (2026), version 1.0
Software baseline	RASS repository, November 2026 (backend ASP.NET Core 10; frontend React 19; forecasting service Python 3.13)
Evaluation data	WFP VAM Food Prices – Rwanda, Humanitarian Data Exchange, licence CC BY-IGO, accessed 1 November 2026

Version	Date	Change
1.0	2026	Executive research brief prepared for MINAGRI and RAB.
1.1	23 July 2026	Live system demonstration at the Rwanda Agricultural Show (AgriShow), Gasabo District, Kigali: national price intelligence, district-level price regulation and 30-day ensemble price forecasting.
2.0	November 2026	Full technical report. Independent code audit; correction of forecasting defects; first out-of-sample evaluation on real Rwandan retail prices; revision of the forecasting ensemble on the basis of that evaluation; security and readiness assessment; pilot roadmap.

Suggested citation

Iriza Gatera, M., Atete Muhire, G. V., Gaju, S., & Ngabonziza, H. (2026). *Rwanda Agri Stability System (RASS): Technical Report, version 2.0*. Faculty of Information Technology, Adventist University of Central Africa. Prepared for the Ministry of Agriculture and Animal Resources (MINAGRI) and the Rwanda Agriculture and Animal Resources Development Board (RAB). Zenodo. https://doi.org/10.5281/zenodo.23870262

Statement on data and results

All forecast accuracy figures in this report were produced by the evaluation script distributed with the software (`forecasting-service/evaluation/backtest_wfp.py`) on publicly available World Food Programme price data, using only information available at each forecast date. Tables and figures are generated directly from the script outputs. Earlier internal dashboards of the platform displayed model "training" metrics that were simulated rather than measured; those components have been removed and replaced (Section 9.1). No figure from those dashboards is used in this report, and none should be cited.

Disclaimer

Acknowledgements

The authors thank their supervisor, Mr. Laurent Maniraho, and the Faculty of Information Technology at AUCA. They also thank the MINAGRI and RAB stakeholders who engaged with the platform during the 2026 presentations, and the visitors to the RASS stand at AgriShow 2026 whose questions shaped this revision. Retail price data were made available by the World Food Programme through the Humanitarian Data Exchange.

Contents

Document control i

Abbreviations viii

Executive summary 1

1 Introduction 4

1.1 Context . . . 4
1.2 Problem statement . . . 4
1.3 Objectives . . . 4
1.4 Scope of this report . . . 5
1.5 Relationship to earlier work . . . 5
1.6 Structure of the report . . . 5

2 System overview 6

2.1 Users and roles . . . 6
2.2 Value chain supported . . . 6
2.3 Rwanda-specific reference data . . . 6

3 Architecture 8

3.1 Components . . . 8
3.2 Technology stack . . . 8
3.3 Data model . . . 8
3.4 Application programming interface . . . 9
3.5 Real-time communication . . . 9
3.6 Integration status . . . 10

4 Functional modules 12

4.1 National price intelligence . . . 12
4.1.1 Price collection . . . 12
4.1.2 Indicators . . . 12
4.2 District-level price regulation . . . 12
4.2.1 Defining a regulation . . . 13
4.2.2 Resolution of the applicable regulation . . . 13
4.2.3 Enforcement and monitoring . . . 13

4.3 Trade, contracts and payments . . . 13
4.3.1 From lot to order . . . 13
4.3.2 Six-step contract . . . 14
4.3.3 Farmer payment allocation . . . 14
4.4 Logistics and tracking . . . 14
4.5 Storage and warehouses . . . 15
4.6 Inter-cooperative crop sharing . . . 15
4.7 Government dashboard and reporting . . . 15
4.8 Conversational assistant . . . 16
4.9 Languages and accessibility . . . 16

5 Forecasting methodology **17**
5.1 Design principles . . . 17
5.2 Inputs . . . 17
5.3 Component models . . . 17
5.4 Ensemble combination . . . 18
5.5 Uncertainty interval . . . 18
5.6 Decision rules and advice . . . 18
5.7 District projections . . . 19
5.8 Model evaluation inside the platform . . . 19

6 Evaluation on WFP Rwanda market prices **20**
6.1 Purpose . . . 20
6.2 Data . . . 20
6.2.1 Source . . . 20
6.2.2 Selection . . . 20
6.3 Evaluation protocol . . . 21
6.3.1 Rolling-origin design . . . 21
6.3.2 Monthly adaptation . . . 21
6.3.3 Models compared . . . 22
6.3.4 Metrics . . . 22
6.3.5 Significance test . . . 22
6.4 Results . . . 22
6.4.1 Overall accuracy . . . 22
6.4.2 Accuracy by horizon . . . 23
6.4.3 Accuracy by crop . . . 24
6.4.4 Robustness: refugee-camp and host-community markets . . . 24

6.4.5 Reliability of the uncertainty intervals . . . 25
6.5 Why Gradient Boosting and LSTM-lite failed . . . 25
6.6 Revision of the production ensemble . . . 26
6.7 Interpretation for decision-makers . . . 27

7 Quality assurance and testing 29
7.1 Automated tests . . . 29
7.2 Reproducible evaluation . . . 29
7.3 Limits of the current test coverage . . . 30

8 Security, privacy and data governance 31
8.1 Controls in place . . . 31
8.2 Findings and remediation plan . . . 31
8.3 Personal data protection . . . 31
8.4 Data governance for price information . . . 32

9 Limitations and corrections 34
9.1 Corrections made in version 2.0 . . . 34
9.2 Remaining limitations . . . 34

10 Pilot roadmap and recommendations 36
10.1 Proposed phases . . . 36
10.2 Pilot success indicators . . . 36
10.3 Recommendations . . . 36

11 Conclusion 38

References 39

A Reproducibility 40
A.1 Software environment . . . 40
A.2 Commands . . . 40
A.3 Series used in the evaluation . . . 40

B API endpoints by controller 42

C Glossary 43

List of Figures

2.1 Value chain and information flows supported by RASS. Blue: market intelligence and policy layer; dashed arrows show price bands constraining listings and forecasts informing buying and selling decisions. 7

3.1 Logical architecture of RASS. 8

6.1 Retail price records per year in the WFP Rwanda dataset. Collection was sparse in 2016–2017; the evaluation uses 2018 onwards. 20

6.2 Mean absolute percentage error by method (lower is better). Dashed line: naive benchmark. 23

6.3 Skill relative to the naive forecast by horizon. Above zero: more accurate than naive. 24

6.4 MAPE by crop for the naive benchmark, SARIMA and the two ensembles. 25

6.5 Coverage (left; dashed line: nominal 80%) and median width (right) of the forecast intervals. 27

6.6 Out-of-sample one-month-ahead forecasts of the statistical ensemble with its 80% interval, against observed WFP retail prices and the naive forecast, for one maize series. 28

List of Tables

2.1 User roles and their principal functions 6

3.1 Principal technologies 9
3.2 Data model by domain 10
3.3 Status of external integrations (November 2026) 11

4.1 Market indicators computed by RASS 12

6.1 Out-of-sample accuracy, all horizons pooled (2,016 forecasts per method) 23
6.2 Skill relative to the naive forecast, by horizon 24
6.3 MAPE (%) by crop, all horizons pooled (best value in bold) 25
6.4 MAPE (%) by market type, all horizons pooled 26
6.5 Empirical coverage and median width of the forecast intervals 26
6.6 Mean ensemble weights during the evaluation 26

7.1 Automated test inventory (all passing) 29

8.1 Security findings and planned remediation 32

9.1 Defects found by the audit and corrections applied 34

10.1 Proposed path from prototype to supervised pilot 36
10.2 Proposed indicators for the supervised pilot 36

A.1 The 28 WFP retail price series retained for the evaluation 40

B.1 Number of HTTP endpoints per backend controller (total 437) 42

Abbreviations

API	Application Programming Interface
AUCA	Adventist University of Central Africa
CSV	Comma-Separated Values
EF Core	Entity Framework Core (object–relational mapper for .NET)
GBR	Gradient Boosting Regressor
HDX	Humanitarian Data Exchange
JWT	JSON Web Token
LLM	Large Language Model
LSTM	Long Short-Term Memory (recurrent neural network)
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
MASE	Mean Absolute Scaled Error
MINAGRI	Ministry of Agriculture and Animal Resources
MoMo	MTN Mobile Money
NCSA	National Cyber Security Authority
OTP	One-Time Password
RAB	Rwanda Agriculture and Animal Resources Development Board
RASS	Rwanda Agri Stability System
RBAC	Role-Based Access Control
RMSE	Root Mean Squared Error
RWF	Rwandan franc
SARIMA	Seasonal AutoRegressive Integrated Moving Average
SMS	Short Message Service
TOTP	Time-based One-Time Password
VAM	Vulnerability Analysis and Mapping (WFP)
WFP	World Food Programme

Executive summary

Purpose. This report documents the Rwanda Agri Stability System (RASS) for the Ministry of Agriculture and Animal Resources (MINAGRI) and the Rwanda Agriculture and Animal Resources Development Board (RAB). It describes what the platform does, how it is built, how its price forecasts are produced, and how accurate those forecasts are on real Rwandan market prices. It also sets out what must be completed before a supervised pilot. It replaces the 2026 executive research brief.

What RASS is. RASS is a web platform that places the main actors of an agricultural value chain on one system: farmers, cooperative managers, buyers, market agents, transporters, storage operators, government officials and administrators. It combines three capabilities that are usually handled by separate tools:

1. **National price intelligence.** Market agents submit prices from registered markets. Government users see national and district price trends, volatility, supply and demand balances, and automatic early warnings.
2. **District-level price regulation.** Government users set minimum and maximum prices per crop for a province, district or market and a period. The platform enforces these bands when cooperatives publish listings, automatically flags out-of-band price submissions, and notifies affected cooperatives in real time.
3. **Price forecasting.** A forecasting service produces forecasts of up to 30 days with uncertainty intervals and role-specific advice (for example "hold" for a seller, "buy early" for a buyer, "monitor" for government).

It also supports the trade cycle from lot submission through listing, ordering, a six-step contract signed by both parties, transport assignment with live GPS tracking, warehouse rental, and farmer payment allocation.

Scale of the software. The platform comprises 34 backend controllers exposing 437 API endpoints over 53 database tables. The web application has 38 pages in English and Kinyarwanda. The forecasting service is written in Python. The software has 78 automated backend tests and 11 forecasting-service tests, all passing.

Key findings of the forecast evaluation

The forecasting models were evaluated for the first time on real data: 28 retail price series from the World Food Programme (WFP) for 6 staple crops in 10 Rwandan markets, January 2018 to July 2026. The evaluation used a rolling-origin design over the last 24 months of each series, giving 2,016 out-of-sample forecasts per model at horizons of one to three months.

- A naive forecast ("next month's price equals this month's") has a mean absolute percentage error (MAPE) of 11.1%. This is the benchmark any useful model must beat.
- The ensemble originally deployed, with five models, reached 10.9% MAPE. It was not reliably better than the naive benchmark. Two of its members, Gradient Boosting and LSTM-lite, had errors of about 29%.
- The revised **statistical ensemble** (SARIMA, Prophet and Holt, weighted by recent accuracy) reaches **9.6% MAPE**. It reduces the error of the naive benchmark by **15.4% two months ahead** and **22.1% three months ahead**. Both reductions are statistically significant ($p < 0.001$), with improvements on 25 and 26 of the 28 series. One month ahead it is not distinguishable from the naive benchmark.
- Its 80% uncertainty interval contains the observed price in 84–88% of cases and is markedly narrower than before.
- Results are consistent across crops and across host-community and refugee-camp markets. The revised ensemble is now the production configuration of the platform.

Corrections made in this version. The independent audit behind this report found several defects. An administrative "model training" screen displayed simulated metrics. A performance endpoint adjusted the ensemble's figures arithmetically and mislabelled two models. A defect caused one forecast model to be reused across different crops and markets. Forecasts also used price submissions that moderation had flagged. All of these have been corrected and covered by tests (Section 9.1). The accuracy figures in this report are the only ones that should be cited.

Readiness. RASS is a *pilot-ready prototype*, not a production system. The software is functionally broad and the price regulation and forecasting components are working and evaluated. Before any field pilot, the following must be completed:

- replacement of the synthetic demonstration price history with verified market data;
- production integrations for SMS and mobile-money disbursement, which are currently stubs or test mode;
- a set of security hardening measures, chiefly removal of credentials from the code repository, authentication rate limiting and protection of uploaded identity documents;
- a deployment configuration and a data-protection review under Law N° 058/2021.

Recommendations to MINAGRI and RAB.

1. Support a three-month hardening phase followed by a six-month supervised pilot in two to three districts, with named MINAGRI/RAB counterparts for price regulation and market monitoring (Chapter 10).
2. Facilitate access to official market price series (for example RAB or NISR price collection) so that forecasts can be trained and re-evaluated on the same data that policy uses.
3. Adopt the evaluation protocol in this report as the acceptance test for any forecasting model used in a government decision tool: out-of-sample, against a naive benchmark, with published error metrics.

4. Treat the platform's food-security and regional scores as indicative dashboards, not official statistics, until their definitions are validated with MINAGRI and NISR.

1 Introduction

1.1 Context

Agriculture is the main source of livelihood for most Rwandan households and a central pillar of national food security. Production is organised around three agricultural seasons: Season A (September to February), Season B (March to June) and Season C (July to August). Harvest and lean periods therefore follow a predictable annual rhythm. Their effect on market prices differs from crop to crop, market to market and year to year.

Price movements matter to every actor in the value chain. A farmer deciding when to sell, a cooperative deciding whether to store, a buyer planning procurement and a ministry deciding whether to intervene all depend on knowing current prices and how they are likely to evolve. In practice, this information is fragmented. Prices are collected by different institutions at different frequencies, trade transactions are recorded on paper or in disconnected tools, and policy instruments such as reference or ceiling prices are communicated through channels that do not reach the point of sale.

National agricultural strategy documents, including the Fifth Strategic Plan for Agriculture Transformation (PSTA 5), give prominence to market access, resilience and the use of digital tools in agriculture. RASS was designed as a contribution to that agenda: a single platform in which market data, trade and price policy meet.

1.2 Problem statement

The project addresses four connected problems:

1. **Price opacity.** Producers and small buyers often transact without reliable knowledge of prevailing prices in nearby markets.
2. **Weak enforcement and visibility of price policy.** When a price band is set for a crop, there is no systematic way to check listed or observed prices against it, or to alert those concerned.
3. **Absence of evaluated forecasts.** Forecasting tools exist, but few are tested out-of-sample on Rwandan data against simple benchmarks. Without such tests, a forecast cannot be trusted for decisions.
4. **Fragmented trade coordination.** Aggregation, sale, contracting, transport, storage and payment are handled separately, which raises transaction costs and post-harvest losses.

1.3 Objectives

The objectives of RASS, and of this report, are:

1. to provide a role-based platform for price collection, market monitoring and trade coordination;
2. to implement district-level price regulation that is enforced at the point of listing and monitored continuously;

3. to produce short-term price forecasts with honest uncertainty, tailored to each role;
4. to evaluate those forecasts rigorously on real Rwandan market prices, and to revise the models on the basis of the evidence;
5. to assess the platform's readiness for a supervised pilot and to state what remains to be done.

1.4 Scope of this report

This report covers the software as it stands in November 2026: functional modules, architecture, data model, forecasting methodology, an out-of-sample evaluation, testing, security, limitations and a pilot roadmap. It does not report a field deployment; none has taken place.

1.5 Relationship to earlier work

An executive research brief (version 1.0) was prepared for MINAGRI and RAB in 2026. The platform was demonstrated at AgriShow 2026 (Gasabo District, 23 July 2026) and presented to MINAGRI stakeholders. Version 2.0 is based on a full independent audit of the source code. It adds the first evaluation on real data and corrects the defects that the audit found (Section 9.1). Where this report differs from earlier material, this report prevails.

1.6 Structure of the report

- Chapter 2 introduces the users and the value chain the platform supports.
- Chapter 3 describes the architecture and the data model.
- Chapter 4 details the functional modules, including price regulation.
- Chapter 5 explains the forecasting methodology.
- Chapter 6 presents the evaluation on WFP data.
- Chapters 7 and 8 cover testing and security.
- Chapter 9 states the limitations and the corrections made.
- Chapter 10 proposes a pilot roadmap and recommendations.

2 System overview

2.1 Users and roles

RASS is organised around eight operational roles, each with its own dashboard and permissions (Table 2.1). An additional *Applicant* role is held temporarily by users who have applied for an operational role and are awaiting administrator approval.

Table 2.1: User roles and their principal functions

Role	Principal functions in RASS
Farmer	View prices and forecasts; submit produce lots to the cooperative; follow contributions and payment allocations; receive price alerts.
Cooperative manager	Register and activate farmers; review lots; manage inventory; publish listings within regulated price bands; accept orders; assign transport; rent warehouse space; request or offer produce to other cooperatives.
Buyer	Browse listings; order and pay through a checkout; sign contracts; follow deliveries; confirm receipt.
Market agent	Submit observed prices from registered markets; review regional comparisons and anomalies; file market reports.
Transporter	Apply for or accept transport jobs; share live location during delivery; record pickup and delivery.
Storage operator	Publish warehouses with capacity and tariffs; manage bookings, rental agreements, goods intake and safety checks.
Government official	Monitor national and district prices, supply and demand, and early warnings; set price regulations and seasonal guidance; moderate price submissions; review contracts and warehouse use; export reports.
Administrator	Manage users, roles and applications; manage reference data and system configuration; review audit logs; run model evaluations.

2.2 Value chain supported

Figure 2.1 summarises the flow of produce, information and money that the platform supports. Each step is recorded in the database, so the same events feed the government dashboards and the forecasting service.

2.3 Rwanda-specific reference data

The platform encodes Rwanda's administrative hierarchy: five provinces (including the City of Kigali), all 30 districts and 402 sectors. Cells and villages are not yet held as master data. Registered markets are linked to their province, district and sector. They are seeded with nine major markets (Kimironko, Nyabugogo, Musanze, Huye, Muhanga, Rwamagana, Nyagatare, Rubavu and Kamembe) and can be extended by government users. A government crop catalogue lists 13 crops (maize, beans, rice, Irish potato, cassava, wheat, sorghum, soybean, banana, tomato, onion, coffee and tea). Crops proposed by other users enter a government review queue. Season A, B and C boundaries are used consistently by the inventory, forecasting and advisory

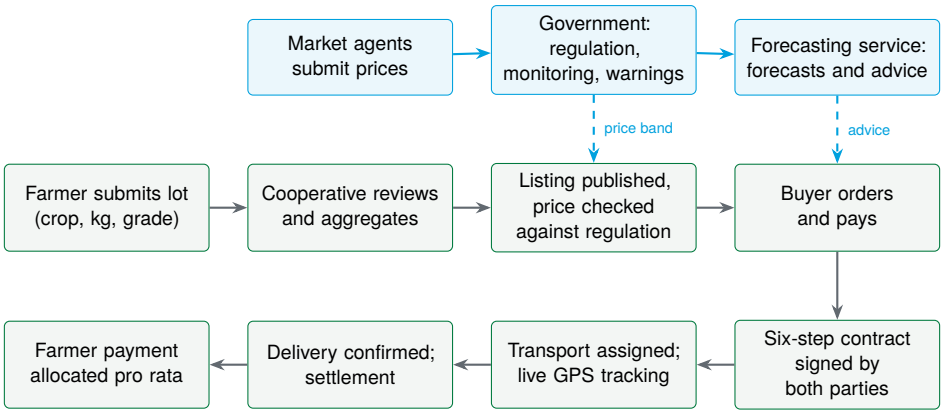


Figure 2.1: Value chain and information flows supported by RASS. Blue: market intelligence and policy layer; dashed arrows show price bands constraining listings and forecasts informing buying and selling decisions.

modules.

3 Architecture

3.1 Components

RASS is built as three services around a relational database (Fig. 3.1):

- **Web application.** A single-page application in React and TypeScript, used in a browser on desktop or mobile.
- **Backend API.** An ASP.NET Core service. It holds the business rules, authentication and authorisation, and the real-time messaging hubs.
- **Forecasting service.** A Python FastAPI service that runs the statistical and machine-learning models. It is called only by the backend and authenticated with a service key.

All persistent data is held in PostgreSQL. The backend calls three external services: an LLM endpoint for the conversational assistant, Stripe Checkout for card payments in test mode, and map tiles from OpenStreetMap for tracking.

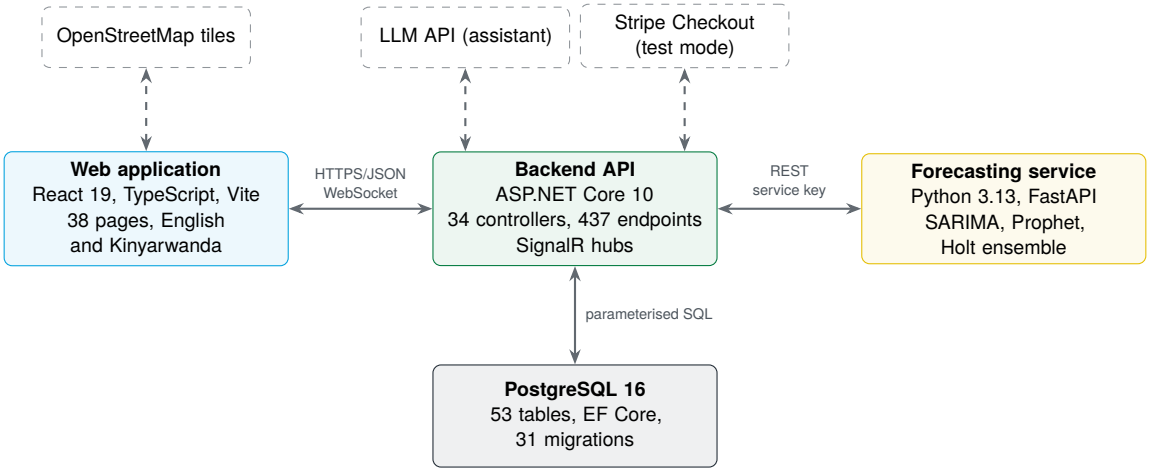


Figure 3.1: Logical architecture of RASS.

3.2 Technology stack

Table 3.1 lists the principal technologies used in each layer. All are open source, except the external services in Section 3.6.

3.3 Data model

The database contains 53 tables, managed through Entity Framework Core migrations created between January and June 2026. Table 3.2 groups them by domain. Every primary key is a UUID. Status fields record the lifecycle of each business object, for example lots, orders, contracts, transport requests and rental agreements.

Each *MarketPrice* record stores:

Table 3.1: Principal technologies

Layer	Technologies
Web application	React 19.2, TypeScript 5.9, Vite 7, TanStack Query 5, Material UI 7, Tailwind CSS 3, Recharts 3, Leaflet / React-Leaflet, i18next, React Hook Form with Zod validation, SignalR client
Backend	ASP.NET Core 10, Entity Framework Core 10 with Npgsql, SignalR, JWT bearer authentication, BCrypt password hashing, Swagger/OpenAPI
Forecasting	Python 3.13, FastAPI, pandas, NumPy, SciPy, statsmodels (SARIMA), Prophet, scikit-learn (Gradient Boosting), custom NumPy recurrent network (LSTM-lite)
Data	PostgreSQL 16
Testing	xUnit, Moq, EF Core in-memory provider (backend); Python `unittest` with FastAPI test client (forecasting)
Packaging	Docker images for each service; Docker Compose definition (to be completed, Chapter 8)

- the crop, the price per kilogram and the observation time;
- the submitting agent and the registered market;
- the market's province, district and sector, copied at submission;
- a verification status (*Approved*, *Pending*, *Flagged* or *Rejected*) with moderation notes.

Prices marked *Flagged* or *Rejected* are excluded from forecasting and from the forecast analytics (national overview, volatility and demand). They remain visible to moderators and to the early-warning service.

3.4 Application programming interface

The backend exposes 437 endpoints across 34 controllers. Responses are JSON with camel-case field names. Each controller maps to a functional area such as authentication, cooperatives, farmers, lots, listings, orders, contracts, payments, transport, tracking, storage, warehouses, government, price regulations, forecasts, reports, administration and reference data. Access is controlled per endpoint by role (Section 8.1). Public endpoints are limited to reference data, marketplace browsing, a three-day forecast preview and registration.

3.5 Real-time communication

Two SignalR hubs provide push communication over WebSockets:

- `/hubs/notifications` delivers in-app notifications (`ReceiveNotification`), chat messages (`ReceiveMessage`) and typing indicators. Each user joins a private group. Price-regulation notices and price alerts are pushed through it.
- `/hubs/tracking` streams transporter positions and delay alerts (events `LocationUpdate` and `DeliveryDelayed`) to the parties of a delivery.

Table 3.2: Data model by domain

Domain	Main tables
Identity and access	Users, Roles, UserRoles, RoleApplications (with messages and documents)
Producers	Cooperatives, Farmers, HarvestDeclarations, Lots, LotContributions, LotImages
Market	MarketListings, ListingImages, CartItems, BuyerProfiles, BuyerOrders
Contracts and finance	Contracts, ContractLots, ContractAttachments, PaymentLedgers, FarmerBalances
Logistics	TransporterProfiles, TransportRequests, TransportJobs, TransportJobApplications, TransporterLocations, DeliveryTrackingInfos
Storage	StorageFacilities, StorageFacilityImages, StorageBookings, StorageBookingMessages, WarehouseRentalRequests, WarehouseRentalAgreements, StorageSafetyChecks, Telemetries
Market intelligence and policy	MarketPrices, RegisteredMarkets, CropCatalogs, PriceRegulations, SeasonalGuidances, MarketReports, CropShareRequests, CropShareBids
Platform	Notifications, ChatMessages, AuditLogs, PlatformAlerts, ModelPerformanceLogs, DataQualityIssues, SystemConfigurations, ScheduledTasks

3.6 Integration status

Table 3.3 states the status of each external integration. It distinguishes components that work end to end from those running in test mode and those that are placeholders. This distinction matters for any pilot decision.

Table 3.3: Status of external integrations (November 2026)

Integration	Status	Notes
Forecasting service	Working	Authenticated service-to-service calls; evaluated in Chapter 6.
Card payments (Stripe Checkout)	Test mode	Checkout sessions and signed webhooks implemented; test credentials only; refunds recorded manually.
Mobile money (MTN MoMo)	Placeholder	Client template without credentials; payouts would require the Disbursements API, not Collections.
Escrow, settlement, payouts	Ledger only	Movements are recorded in the payment ledger; no funds are transferred by the platform.
SMS	Placeholder	Messages are logged; the SMS provider call is not enabled. One-time codes are therefore not yet delivered by SMS.
E-mail	Not implemented	Preferences are stored; no e-mail is sent.
Conversational assistant	Working	Hosted LLM API with role context; requires an API key; no offline fallback.
Maps	Working	OpenStreetMap tiles through Leaflet.
Storage telemetry	Manual	Temperature and humidity are entered during safety checks; no sensor ingestion.

4 Functional modules

4.1 National price intelligence

4.1.1 Price collection

Market agents submit prices for a crop at a *registered* market. The market's province, district and sector are attached to the record automatically, so every price is geographically traceable. Each submission is audit-logged and moderated automatically against any applicable price regulation (Section 4.2). Government users can review, approve or reject submissions, and the submitting agent is notified of the decision.

4.1.2 Indicators

Indicators are computed from the price records in the database. Trend, volatility and the national overview use only submissions that are not flagged or rejected. Early warnings use all observed prices, because out-of-band prices are precisely what they are meant to detect. Table 4.1 lists the main indicators with their definitions. Several of them are heuristic composites designed for monitoring. Their thresholds are configurable design choices, not official statistical definitions, and we recommend that MINAGRI and NISR validate them before they are used in official communication.

Table 4.1: Market indicators computed by RASS

Indicator	Definition
Seven-day trend	Change between the mean of the latest 7 and the previous 7 observations for a crop; *up* above $+2\%$, *down* below -2%.
Volatility	Coefficient of variation (CV) of prices over the last 60 days; low ≤ 0.10, medium ≤ 0.20, high > 0.20.
Price anomaly	$z = (p - \bar{p})/s$ against at least 10 historical observations; anomaly if $\lvert z\rvert > 2.5$, high severity if $\lvert z\rvert > 3.5$.
Price-spike warning	7-day mean price above 1.25 times the 60-day mean (high severity above 1.5 times).
Supply-shortage warning	Total recorded stock of a crop below 1,000 kg (high severity below 200 kg).
Regulation-violation warning	Observed prices outside an active regulation band in the last 7 days (high severity above 5 observations).
Supply–demand ratio	Active listed supply divided by open and accepted order demand; surplus above 1.2, deficit below 0.8.
Food-security index (indicative)	$\min\big(95, \max(40,\ 100 - 8H - 3A)\big)$, where H is the number of high-volatility crops and A the number of crops with price alerts.

4.2 District-level price regulation

Price regulation is the platform's most direct policy instrument. It is restricted to the Government and Administrator roles.

4.2.1 Defining a regulation

A regulation specifies:

- a crop from the government catalogue;
- a geographic scope: a province, and optionally a district or a specific registered market;
- a maximum price per kilogram and an optional minimum;
- an effective period.

The platform validates that the minimum is below the maximum, that the period is in the future and well ordered, and that no active regulation already covers the same crop and scope for an overlapping period. District and market names are normalised against the administrative reference data. If a market is given, its district and province are taken from the market register.

4.2.2 Resolution of the applicable regulation

When a price must be checked, the most specific active regulation applies. For the validation service and price moderation, the order of precedence is market, then district, then province. Listing enforcement currently resolves by the cooperative's district and then its province.

4.2.3 Enforcement and monitoring

The regulation is applied at four points:

1. **At listing.** A cooperative cannot publish a listing whose price lies outside the applicable band. The request is rejected with an explanation of the regulated range.
2. **At price submission.** A market agent's observed price is marked *Approved* if it lies within the band, *Flagged* with a moderation note if it lies outside, and *Pending* if no regulation applies. Flagged prices are excluded from forecasting and analytics.
3. **At creation of a regulation.** Cooperative managers with active listings for the crop receive an in-app notification, delivered in real time over SignalR. Managers whose listing price violates the new band receive a warning.
4. **Continuously.** The early-warning service reports crops with recent observations outside an active band (Table 4.1).

A public validation endpoint lets listing forms check a proposed price before submission. Government users can also publish *seasonal guidance* per crop and region: an expected trend, an expected price range and a recommendation to farmers.

4.3 Trade, contracts and payments

4.3.1 From lot to order

Produce enters the platform in two ways:

- A farmer submits a lot (crop, quantity, grade, expected harvest date), which the cooperative manager approves or rejects with a reason.

- A manager records pooled inventory directly, with each farmer's contribution, which must sum to the lot quantity.

Listings are created from inventory, checked against stock and regulation, and published to the marketplace. Buyers add listings to a cart and pay through a hosted card checkout. A cooperative can accept an order only once it has been paid. Accepting an order reduces the listed quantity, attaches the inventory lots and creates a contract and a transport request.

4.3.2 Six-step contract

Each contract follows six steps: review, signature, escrow, logistics, delivery and settlement. A step is complete only when both the buyer and the seller have confirmed it.

- **Review.** A rejection returns the document for amendment.
- **Signature.** Signing uses a six-digit one-time code.
- **Escrow and settlement.** Funds held and released are recorded in the payment ledger.
- **Disputes.** Disputes are resolved by an administrator: release, refund or partial refund.

Attachments up to 8 MB can be added to a contract.

4.3.3 Farmer payment allocation

When an order is paid, the amount is allocated to the cooperative's farmers in proportion to the quantities they contributed:

$$\text{payout}_f = \text{amount} \times \frac{q_f}{\sum_g q_g}, \tag{4.1}$$

where q_f is the quantity contributed by farmer f to matching inventory of the crop. Contribution records take precedence over lot ownership. Allocations are created as pending when payment is received and marked paid when the cooperative accepts the order. No cooperative commission is deducted in the current version. Allocations are records of entitlement; disbursement through mobile money is not yet connected (Section 3.6).

4.4 Logistics and tracking

A cooperative can assign a transport request in three ways:

- directly to a verified transporter with sufficient capacity;
- through a job posting to which transporters apply with a proposed price and documents;
- by letting an eligible transporter accept it.

During a delivery the transporter's browser shares its position. Each position update is stored and broadcast to the parties. Progress and arrival time are estimated from straight-line distance, and a delay alert is sent when the estimate slips by more than 15 minutes. The default transport price in the current version is a flat placeholder ($10{,}000 + 50$ RWF/kg). Distance-based pricing is part of the roadmap.

4.5 Storage and warehouses

Storage operators publish warehouses with capacity, supported crops, features, tariffs and coordinates. Cooperatives search for space with a ranked discovery service that combines six weighted criteria:

- location, 40%, from GPS distance or the same district or province;
- crop compatibility, 20%;
- capacity, 15%;
- features, 10%;
- availability, 10%;
- value for money, 5%.

A rental proceeds as a request, an optional visit, a draft agreement, signature by both parties, payment of the first rent, and activation. Available capacity is recomputed from active and pending commitments, so it is never allocated twice. Goods intake is permitted only on active, paid agreements and within the allocated capacity. A cooperative can delegate full stock management to the operator under a separate delegation contract. Safety checks record temperature, humidity, pest and structural status. Stock ageing is classified in five bands, from 0–7 days to more than 60 days.

4.6 Inter-cooperative crop sharing

A cooperative facing a shortage can broadcast a request for a crop, or address it to one cooperative. Other cooperatives with sufficient stock submit bids, and the requester selects one. A regional supply balance shows where surpluses and shortages lie: the ratio of listed supply to demand is classed as surplus above 1.5 and scarcity below 0.7. In the current version, acceptance and fulfilment are recorded as status changes. Generating the contract and transport request automatically is a roadmap item.

4.7 Government dashboard and reporting

The government dashboard brings the policy functions together in 17 sections:

- overview, price trends and price moderation;
- regulations and seasonal guidance;
- early warnings, supply and demand, and field reports;
- cooperatives, farmers and transporters;
- contracts and warehouse rentals;
- reports, security, an access centre and the AI assistant.

Eighteen report types can be generated on screen. About 25 role-filtered datasets can be exported as CSV files that open directly in spreadsheet software. Examples include regulation compliance, supply and demand, regional performance, price trends and contracts. PDF export is not yet available.

4.8 Conversational assistant

Each dashboard includes an assistant that answers questions using a hosted large language model. The model is instructed to ground its answers in the platform context passed with each question and to state missing data rather than invent numbers. It replies in the language of the question, English or Kinyarwanda. The assistant is an aid to navigation and interpretation; it does not take actions in the platform.

4.9 Languages and accessibility

The web application is available in English and Kinyarwanda. The interface catalogue holds 1,282 English and 1,213 Kinyarwanda strings, so about 95% of strings are translated. Server messages for authentication are localised. The interface adapts to mobile screens and offers light and dark themes. Farmers can sign in with their phone number. Their accounts are created and activated by the cooperative with a one-time activation code.

5 Forecasting methodology

5.1 Design principles

The forecasting service follows four principles that emerged from this revision:

1. **One validated model.** Every forecast shown to any user comes from the same ensemble, the one evaluated in Chapter 6.
2. **Benchmark against naive.** A model is useful only if it beats the forecast "tomorrow's price equals today's" out of sample. That benchmark is reported alongside every evaluation.
3. **Honest uncertainty.** Each forecast carries an interval calibrated on the model's own recent errors. Without enough history to estimate those errors, the advice defaults to *Monitor*.
4. **Real data only.** Forecasts use verified platform prices. Flagged and rejected submissions are excluded. The data source and number of observations are returned with each forecast.

5.2 Inputs

For a request (crop, market, horizon $H \le 30$ days), the backend proceeds as follows:

1. It retrieves the 90 most recent verified prices for the crop at that market.
2. If fewer than seven are available, it falls back to the crop's national series, and then to recent listing prices, and states which source was used.
3. With fewer than seven observations in total, it declines to forecast.

The series is passed to the forecasting service, which returns daily forecasts and intervals.

5.3 Component models

Let $y_1, \dots, y_n$ be the price series and $\hat{y}_{n+h}$ the forecast h steps ahead.

Holt linear exponential smoothing. A level ℓ_t and trend b_t are updated recursively (Holt, 2004):

$$\ell_t = \alpha y_t + (1-\alpha)(\ell_{t-1} + b_{t-1}), \qquad b_t = \beta(\ell_t - \ell_{t-1}) + (1-\beta) b_{t-1}, \qquad \hat{y}_{n+h} = \ell_n + h\, b_n, \tag{5.1}$$

with $\alpha = 0.3$ and $\beta = 0.1$.

SARIMA. A seasonal ARIMA$(1,1,1)(0,1,1)_s$ model (Box et al., 2015) is estimated by maximum likelihood with `statsmodels` (Seabold and Perktold, 2010). The seasonal period is $s = 7$ for daily platform prices, capturing weekly market-day effects, and $s = 12$ for monthly data in the evaluation. If estimation fails or the series has fewer than 14 points, the Holt forecast is used.

Prophet. An additive model of piecewise-linear trend and Fourier seasonality (Taylor and Letham, 2018). The changepoint prior scale is 0.05, which limits over-reaction to noise. Weekly seasonality is enabled for daily series longer than 30 points and yearly seasonality for series longer than a year, or for monthly data.

Diagnostic models (zero weight). Two further models are still computed and displayed for transparency, but receive no weight in the ensemble (Section 6.6):

- **Gradient Boosting.** A gradient-boosted regression-tree model (Friedman, 2001) on four engineered features: one-step and seven-step percentage changes, momentum (the difference between 7- and 14-step moving averages) and the 14-step coefficient of variation.
- **LSTM-lite.** A single-layer gated recurrent network with eight hidden units, implemented in NumPy (Elman, 1990; Hochreiter and Schmidhuber, 1997).

Both models forecast recursively, feeding their own predictions back as inputs.

5.4 Ensemble combination

Combining forecasts reduces the risk of relying on a single misspecified model (Bates and Granger, 1969). Large forecasting competitions have repeatedly found combinations of statistical methods to be hard to beat (Makridakis et al., 2020). The RASS ensemble combines SARIMA, Prophet and Holt with weights inversely proportional to each model's root mean squared error on a hold-out at the end of the series:

$$w_k = \frac{1/\mathrm{RMSE}_k}{\sum_j 1/\mathrm{RMSE}_j}, \qquad \hat{y}^{\,\mathrm{ens}}_{n+h} = \sum_k w_k \, \hat{y}^{\,(k)}_{n+h}, \tag{5.2}$$

where the hold-out length is $m = \min\bigl(7, \max(3, \lfloor n/5 \rfloor)\bigr)$. The models are fitted on $y_1, \dots, y_{n-m}$ to compute the weights, then refitted on the full series to forecast. With fewer than ten observations, equal weights are used.

5.5 Uncertainty interval

The interval uses the hold-out error of the *combined* forecast, $\hat{\sigma}$, widened with the horizon:

$$\bigl[\hat{y}^{\,\mathrm{ens}}_{n+h} - z\,\hat{\sigma}\sqrt{h}, \;\; \hat{y}^{\,\mathrm{ens}}_{n+h} + z\,\hat{\sigma}\sqrt{h}\bigr], \qquad z = 1.2816, \tag{5.3}$$

which is a nominal 80% interval under a random-walk error growth assumption. It replaces an earlier interval based on the spread between models, which was wide and had no stated coverage (Section 6.4.5).

5.6 Decision rules and advice

The forecast is translated into a recommendation from the expected change $\Delta = (\bar{\hat{y}} - y_n)/y_n$ over the horizon:

- *Hold* if $\Delta > +5\%$ and *Sell now* if $\Delta < -5\%$. The thresholds widen to $\pm 8\%$ when recent volatility is high, that is when the coefficient of variation of the last 30 observations exceeds 0.25.

- *Monitor* otherwise.
- *Monitor* also whenever the model confidence is below 0.6. Confidence is defined as $1 - \mathrm{MAPE}$ of the ensemble on the hold-out.

The trend is classified from the slope of the forecast path: *up* if it exceeds $+0.5\%$ of the price level per step, *down* below -0.5%. The same forecast yields different advice for each role:

- a seller is advised on timing of sales;
- a buyer on timing of purchases;
- a transporter on expected demand for transport;
- a government user on whether prices are approaching an intervention band of $\pm 15\%$ around the current price.

5.7 District projections

For government and other non-public users, the market forecast is projected to all 30 districts. The projected price for district d is:

$$\hat{p}_d = \bar{p}_d \times \frac{\hat{y}^{\mathrm{ens}}_{n+H}}{\hat{y}^{\mathrm{ens}}_{n+1}} \times \mathrm{clip}\left(\frac{p_d^{\mathrm{last}}}{p_d^{\mathrm{first}}},\, 0.85,\, 1.15 \right), \tag{5.4}$$

where $\bar{p}_d$ is the district's mean verified price over 60 days and the last factor is the district's own momentum. Districts without observations use their province's values. The displayed confidence falls from at most 0.95 for well-observed districts to 0.45 for districts without data. These projections are a transparent scaling of the evaluated forecast, not a separately validated model, and they are labelled as such.

5.8 Model evaluation inside the platform

Administrators can run a rolling-origin evaluation on any crop and market from the administration dashboard. The service performs the following steps:

1. It cuts the verified history at ten successive points.
2. At each cut-off, it fits every model on the data before that point and scores it on the next seven observations.
3. It reports MAE, RMSE, MAPE, the share of forecasts within $\pm 10\%$ and the skill against the naive forecast.

Results are stored in the model performance log. A model that does worse than naive is marked *Degraded*. This replaces the earlier simulated training screen.

6 Evaluation on WFP Rwanda market prices

6.1 Purpose

A forecast is useful to a decision-maker only if it is more accurate than what the decision-maker could obtain without it. This chapter therefore answers three questions:

1. How accurate are the RASS models on real Rwandan retail prices, out of sample?
2. Are they more accurate than simple benchmarks?
3. Are their uncertainty intervals reliable?

The price history in the platform's demonstration database is synthetic, so it cannot answer these questions. We therefore used the World Food Programme's public price monitoring data for Rwanda.

6.2 Data

6.2.1 Source

The source is the WFP VAM Food Prices dataset for Rwanda, distributed through the Humanitarian Data Exchange under the CC BY-IGO licence (World Food Programme, 2026). The file accessed on 1 November 2026 contains 153,204 price records from 109 markets in all five provinces, January 2000 to August 2026. Almost all of them (99.6%) are retail prices in Rwandan francs.

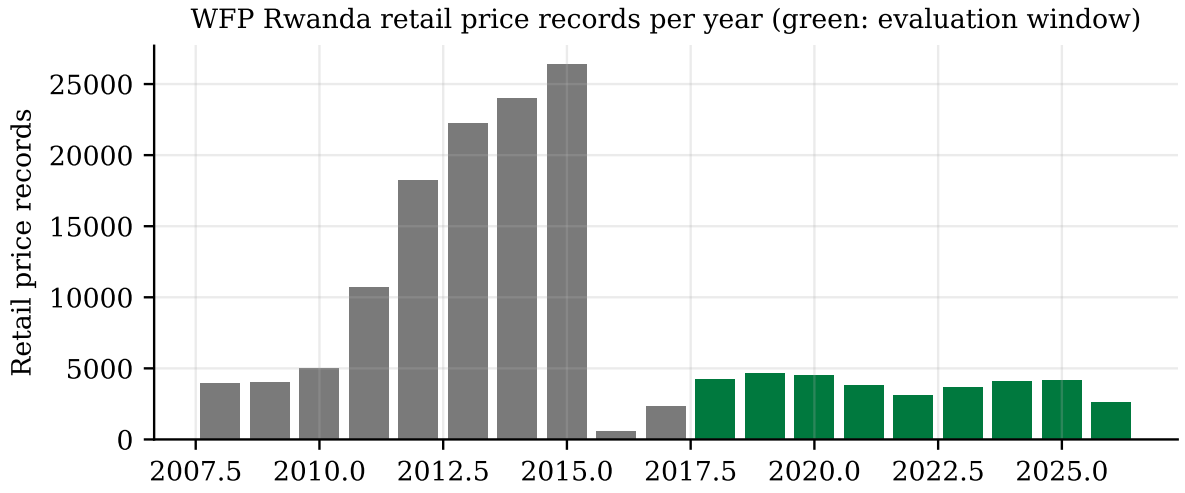


Figure 6.1: Retail price records per year in the WFP Rwanda dataset. Collection was sparse in 2016–2017; the evaluation uses 2018 onwards.

6.2.2 Selection

We kept retail prices in RWF/kg flagged as actual observations for the six RASS crops with continuous recent coverage. In WFP terminology these are maize, dry beans, Irish potatoes,

cassava, sorghum and local rice. Tomatoes and wheat, also covered by RASS, have no WFP retail series after 2015 and are excluded. Observations were aggregated to one value per market and month (median). A series was retained if it met three conditions:

1. it starts in or after January 2018, after the 2016–2017 collection gap shown in Fig. 6.1;
2. at least 85% of its months are observed;
3. it has at least 60 months of data after filling.

Interior gaps of up to three months were filled by linear interpolation (352 points in total). Series with longer gaps were truncated after the last long gap. This gives **28 series** for 6 crops in 10 markets, covering January 2018 to July 2026 (Table 6.3).

Representativeness

The strict coverage rule keeps only markets that WFP monitors continuously. These are in the Southern, Eastern and Western provinces, and 13 of the 28 series come from markets serving refugee camps. No series from the City of Kigali or the Northern Province met the rule. Section 6.4.4 shows that the conclusions are the same for camp and host-community markets. They should still be confirmed on official MINAGRI or RAB price series covering all provinces before national use.

6.3 Evaluation protocol

6.3.1 Rolling-origin design

We used a rolling-origin (expanding-window) design, the standard out-of-sample procedure for forecast evaluation (Tashman, 2000; Hyndman and Athanasopoulos, 2021):

1. For each series we take the last 24 monthly cut-off points.
2. At each cut-off, every model is fitted on the data up to that month only.
3. It then forecasts one, two and three months ahead.
4. The forecasts are compared with the prices WFP later observed.

No information from after a cut-off is used to produce its forecast, including the ensemble weights. Each model therefore produces 2,016 out-of-sample forecasts: 28 series, 24 cut-offs and three horizons.

6.3.2 Monthly adaptation

Because the public data are monthly, the models were evaluated at that frequency:

- SARIMA uses period $s = 12$;
- Prophet uses yearly seasonality on month-start dates;
- the Holt, Gradient Boosting and LSTM-lite models run on monthly steps.

Model structures and hyper-parameters are otherwise identical to those in the platform. The platform's daily forecasts (horizons up to 30 days) are therefore supported by this evaluation in method, but not yet in measured accuracy at the daily scale. The in-platform evaluation tool (Chapter 5) is intended for that purpose once verified daily data are collected.

6.3.3 Models compared

Ten forecasting methods were compared:

- two benchmarks: *naive* (the last observed price) and *seasonal naive* (the price twelve months earlier);
- the five component models: Holt, SARIMA, Prophet, Gradient Boosting and LSTM-lite;
- three combinations:
  - the *original 5-model ensemble* as deployed before this revision;
  - an *equal-weight ensemble* of the same five models;
  - the *statistical ensemble* of SARIMA, Prophet and Holt, with inverse-RMSE weights.

6.3.4 Metrics

For actual prices y and forecasts $\hat{y}$ we report:

- MAE, the mean absolute error in RWF/kg;
- RMSE, the root mean squared error;
- MAPE, the mean absolute percentage error;
- MASE, the MAE scaled by the mean absolute one-step change of each training window (Hyndman and Koehler, 2006);
- the share of forecasts within $\pm 10\%$ of the actual price.

The *skill* of a model at horizon h is the percentage reduction of MAE relative to the naive forecast:

$$\text{skill}_h = 100 \times \left(1 - \text{MAE}_h^{\text{model}} / \text{MAE}_h^{\text{naive}}\right).$$

6.3.5 Significance test

To test whether a difference from the naive forecast is systematic, we follow the logic of the Diebold–Mariano test (Diebold and Mariano, 1995):

1. For every forecast we compute the loss differential $d = |e^{\text{model}}| - |e^{\text{naive}}|$.
2. We average d within each series, so that the 28 series are treated as independent units. This avoids overstating significance through autocorrelation within a series.
3. We test whether the mean across series differs from zero with a two-sided t-test.

We also report the number of series on which the model beats naive.

6.4 Results

6.4.1 Overall accuracy

Table 6.1 and Fig. 6.2 summarise accuracy over all horizons. SARIMA and the statistical ensemble are the only methods with a clearly lower error than the naive forecast: MAPE 9.5% and 9.6%, against 11.1% for naive. They also have the highest share of forecasts within $\pm 10\%$,

61% for the statistical ensemble against 54% for naive. The original 5-model ensemble is only marginally better than naive overall (10.9%). Gradient Boosting and LSTM-lite are far worse than every benchmark, at 28.8% and 29.0%.

Table 6.1: Out-of-sample accuracy, all horizons pooled (2,016 forecasts per method)

Method	MAE	RMSE	MAPE (%)	MASE	Within ±10% (%)
Naive (last value)	77.4	101.2	11.13	1.71	54.3
Seasonal naive	135.6	177.5	18.80	2.86	34.8
Holt linear	93.2	126.2	13.17	2.04	50.3
SARIMA$(1,1,1)(0,1,1)_{12}$	66.6	87.7	9.48	1.47	62.0
Prophet	91.6	114.7	13.92	2.04	46.3
Gradient Boosting	211.2	255.7	28.77	4.71	17.5
LSTM-lite	220.0	244.9	29.01	4.92	4.5
Equal-weight ensemble	93.9	117.7	12.52	2.11	43.3
Original 5-model ensemble	80.5	105.7	10.93	1.77	55.4
Statistical ensemble	67.1	86.2	9.64	1.49	60.8

MAE and RMSE in RWF/kg. Statistical ensemble = SARIMA + Prophet + Holt, inverse-RMSE weights.

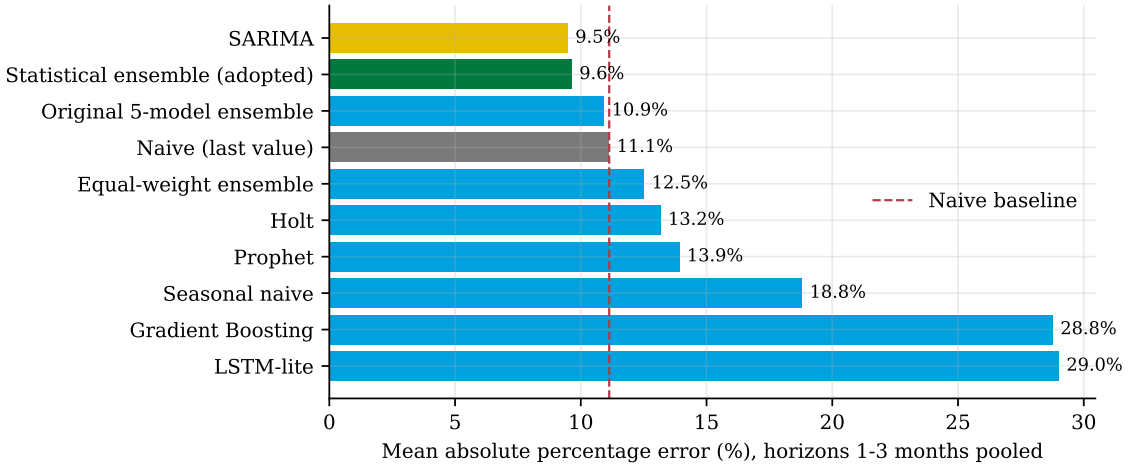


Figure 6.2: Mean absolute percentage error by method (lower is better). Dashed line: naive benchmark.

6.4.2 Accuracy by horizon

Table 6.2 and Fig. 6.3 show how skill depends on the horizon.

- **One month ahead.** The last observed price is very hard to beat, and no method improves significantly on it. SARIMA gains 3.3% ($p = 0.21$). The statistical ensemble is -5.0% ($p = 0.07$), a difference not statistically distinguishable from zero.
- **Two and three months ahead.** The statistical ensemble reduces the naive error by 15.4% and 22.1%. Both reductions are significant ($p < 0.001$), and the ensemble improves on naive for 25 and 26 of the 28 series respectively.

SARIMA alone is close (14.3% and 19.9%). The ensemble's advantage over SARIMA is its robustness: it never relies on a single model whose estimation may fail on a particular series.

Table 6.2: Skill relative to the naive forecast, by horizon

	1 month			2 months			3 months		
Method	Skill %	p	Better	Skill %	p	Better	Skill %	p	Better
Seasonal naive	-159.7	< 0.001	0/28	-67.3	< 0.001	4/28	-33.8	< 0.001	6/28
Holt linear	-41.9	< 0.001	1/28	-17.8	< 0.001	5/28	-10.2	< 0.001	7/28
SARIMA$(1,1,1)(0,1,1)_{12}$	+3.3	0.206	16/28	+14.3	< 0.001	23/28	+19.9	< 0.001	26/28
Prophet	-72.5	< 0.001	0/28	-12.9	0.015	10/28	+7.7	0.138	15/28
Gradient Boosting	-193.1	< 0.001	0/28	-171.2	< 0.001	0/28	-162.6	< 0.001	0/28
LSTM-lite	-287.2	< 0.001	0/28	-172.8	< 0.001	0/28	-135.2	< 0.001	0/28
Equal-weight ensemble	-45.5	< 0.001	0/28	-19.5	< 0.001	8/28	-9.1	0.070	13/28
Original 5-model ensemble	-22.5	< 0.001	2/28	-3.5	0.372	14/28	+6.3	0.094	21/28
Statistical ensemble	-5.0	0.073	10/28	+15.4	< 0.001	25/28	+22.1	< 0.001	26/28

Skill: % reduction of MAE relative to naive (positive is better). p: two-sided t-test on per-series mean loss differentials ($n = 28$ series). Better: number of series on which the method has lower MAE than naive.

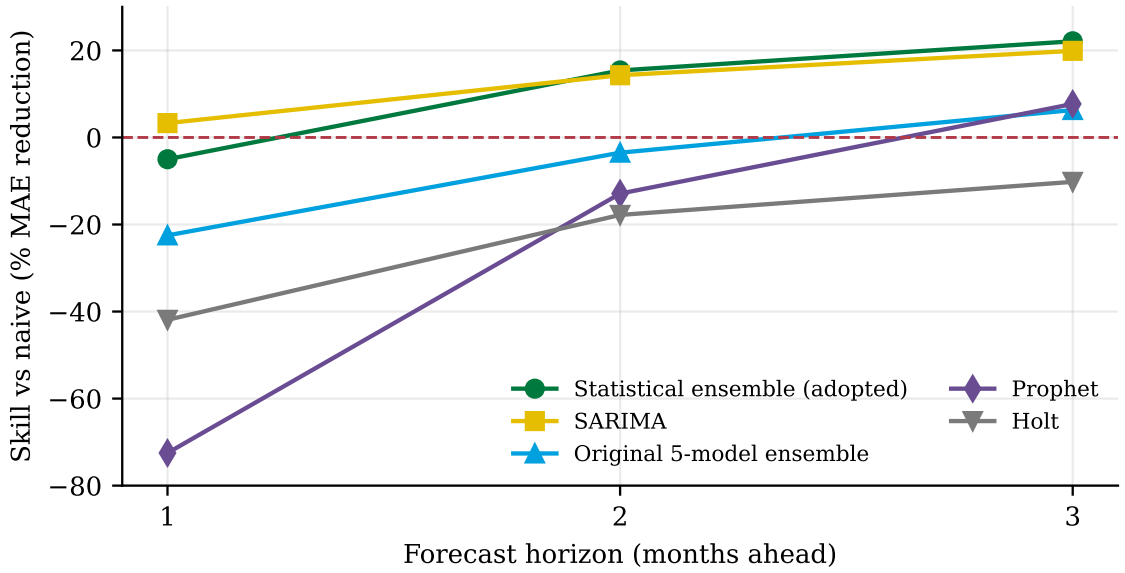


Figure 6.3: Skill relative to the naive forecast by horizon. Above zero: more accurate than naive.

6.4.3 Accuracy by crop

Table 6.3 and Fig. 6.4 break results down by crop. The statistical ensemble or SARIMA is the most accurate method for maize, beans, Irish potatoes and sorghum, the four crops with several series. For cassava and local rice, each represented by a single series, the naive forecast is best, and those results should not be generalised.

6.4.4 Robustness: refugee-camp and host-community markets

Almost half the series come from markets serving refugee camps, whose price dynamics may differ, so we split the results by market type (Table 6.4). The statistical ensemble reduces the naive error by the same proportion in both groups. The ranking of methods is unchanged.

Table 6.3: MAPE (%) by crop, all horizons pooled (best value in bold)

Crop	Series	Naive	SARIMA	Prophet	Original ens.	Statistical ens.
Beans	10	11.0	**9.1**	13.8	10.8	9.2
Cassava	1	**9.3**	10.8	16.8	15.5	12.4
Maize	6	12.6	10.1	13.9	11.3	**9.4**
Potatoes	6	13.3	**11.2**	17.8	11.7	12.1
Rice	1	**4.6**	5.2	9.1	8.0	6.5
Sorghum	4	8.0	7.6	8.9	9.3	**7.5**

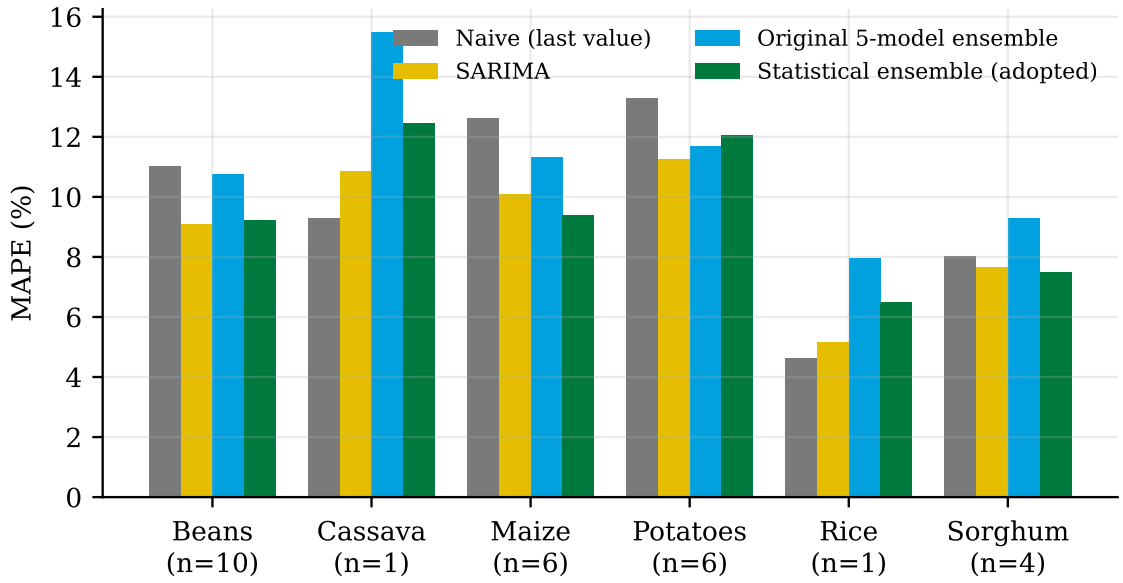


Figure 6.4: MAPE by crop for the naive benchmark, SARIMA and the two ensembles.

6.4.5 Reliability of the uncertainty intervals

An 80% interval should contain the observed price in about 80% of cases, and should be as narrow as possible while doing so. Table 6.5 and Fig. 6.5 compare the two interval methods:

- **Original interval.** Based on the spread between the five models, it covered 90–94% of outcomes. Its median width was about 50% of the price one month ahead, too wide to support a decision.
- **Adopted interval.** Based on the ensemble's own hold-out error, it covers 84–88%, slightly conservative relative to the nominal 80%. It is about 40% narrower at one month (median width 30% of the price).

Figure 6.6 illustrates one-month-ahead forecasts and intervals for one maize series over the evaluation period.

6.5 Why Gradient Boosting and LSTM-lite failed

Both models forecast several steps ahead by feeding their own predictions back as inputs. Their features are short-term percentage changes and moving-average momentum, and these become unreliable after one step: errors compound with every recursive step. Both also extrapolate

Table 6.4: MAPE (%) by market type, all horizons pooled

Market type	Series	Naive	SARIMA	Original ens.	Statistical ens.	Skill (%)
Host-community markets	15	11.37	9.62	10.98	9.76	+13.4
Refugee-camp markets	13	10.85	9.32	10.86	9.49	+13.4

Skill: % reduction of MAE of the statistical ensemble relative to naive.

Table 6.5: Empirical coverage and median width of the forecast intervals

	1 month		2 months		3 months	
Interval method	Cov. %	Width %	Cov. %	Width %	Cov. %	Width %
Original: ±1.15× half-spread of 5 members	94.3	49.5	91.4	53.7	90.0	58.1
Adopted: $\pm z_{0.9}\,\hat{o}_{\text{hold-out}}\sqrt{h}$	84.1	30.2	86.3	41.8	88.4	51.7

Coverage: share of observed prices inside the interval. Width: median interval width as % of the observed price.

poorly. Tree ensembles cannot predict outside the range of their training targets, and the LSTM-lite is trained with one-step back-propagation on a few dozen points. On series of 60 to 100 monthly observations these weaknesses dominate. The finding is consistent with evidence that, on short and noisy series, simple statistical methods and their combinations tend to outperform machine-learning methods (Makridakis et al., 2020, 2018). It does not rule out machine-learning models in general. Direct multi-horizon models trained across many series, with exogenous drivers such as rainfall, are a promising direction once richer data are available (Chapter 10).

6.6 Revision of the production ensemble

On the basis of this evidence the production configuration was changed:

1. The ensemble members are now SARIMA, Prophet and Holt. Gradient Boosting and LSTM-lite remain visible as diagnostics with weight zero.
2. The interval is now the hold-out-error interval.
3. All forecasts, including the role-specific advice, are produced by this ensemble.
4. The confidence used by the decision rules is the ensemble's measured hold-out accuracy.

Table 6.6 shows the average weights the ensembles assigned during the evaluation. Even after inverse-RMSE weighting, the original ensemble gave about 27% of its weight to the two weakest members, which is why it could not improve on naive.

Table 6.6: Mean ensemble weights during the evaluation

Member	Original 5-model ensemble	Statistical ensemble
SARIMA	0.272	0.379
Prophet	0.234	0.317
Holt linear	0.223	0.304
Gradient Boosting	0.124	0.000
LSTM-lite	0.146	0.000

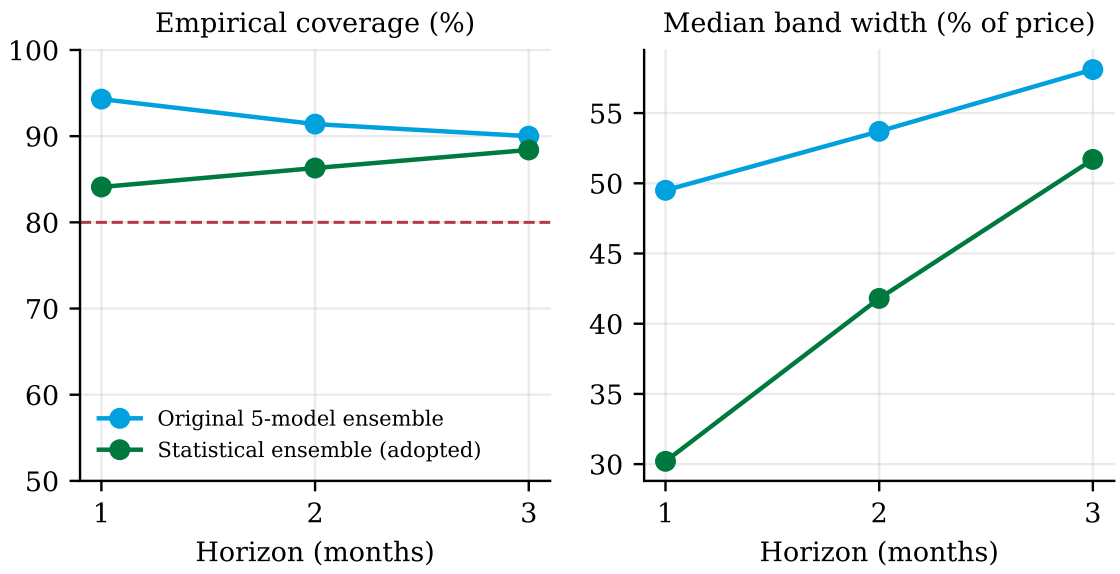


Figure 6.5: Coverage (left; dashed line: nominal 80%) and median width (right) of the forecast intervals.

6.7 Interpretation for decision-makers

- For one-month decisions, the latest verified price is as good a guide as any model. The value of RASS at this horizon lies in making current prices visible and comparable across markets, not in the forecast.
- For two- to three-month decisions, such as storage, procurement planning or anticipating price pressure, the statistical ensemble provides a measurable improvement, with typical errors of about 9–10% of the price.
- Intervals should be read as ranges of plausible outcomes. At one month ahead they span about 30% of the price; at three months, about half of it.

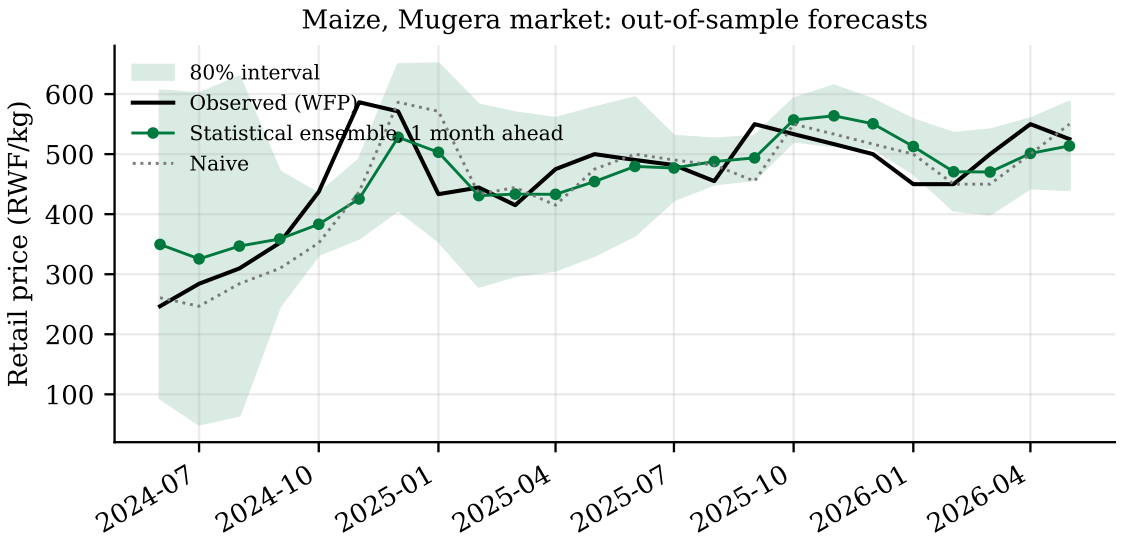


Figure 6.6: Out-of-sample one-month-ahead forecasts of the statistical ensemble with its 80% interval, against observed WFP retail prices and the naive forecast, for one maize series.

7 Quality assurance and testing

7.1 Automated tests

The software includes two automated test suites, both passing on the November 2026 baseline (Table 7.1).

- The **backend suite** (xUnit) runs against an in-memory database, with external services replaced by test doubles.
- The **forecasting suite** (Python `unittest`) exercises the models and the HTTP API of the forecasting service.

Table 7.1: Automated test inventory (all passing)

Category	Cases	What is verified
Backend – unit	31	Authentication (unknown user, valid login with role claims, expired reset codes); token generation; rental payment calculations; warehouse capacity rules; controller catalogue conventions; model-evaluation service (queued state, measured metrics logged, flagged prices excluded, degraded-model flag, failure handling).
Backend – integration	4	Rental payment activation and capacity snapshot; recurring payment; consistency of seeded cooperative data.
Backend – security	16	JWT claims, expiry and rejection of tampered signatures; presence of role restrictions on protected endpoints; one-time-code hashing and expiry; handling of injection-style inputs.
Backend – performance	3	Response-time thresholds for payment calculations, capacity snapshots and an evaluation cycle.
Backend – API surface	24	Every role has the expected controllers and routes.
Forecasting service	11	Ensemble weights (statistical members only, sum to one); interval brackets the forecast and widens with horizon; accuracy derived from hold-out error; independence of successive requests; Rwanda season mapping; API key enforcement; 30-day horizon; short history defaults to *Monitor*; evaluation endpoint and its input checks; performance endpoint serves backtest results.
Total	**89**	

7.2 Reproducible evaluation

The evaluation in Chapter 6 is itself a test of the forecasting component. A single script reads the stored WFP file, applies the selection rules, runs the rolling-origin protocol and writes the summary metrics and every individual forecast error to files. The report's tables and figures are generated from those files by two further scripts, so every number in Chapter 6 can be regenerated (Appendix A).

7.3 Limits of the current test coverage

The test suites do not yet include:

- end-to-end tests through the HTTP pipeline with a real PostgreSQL database;
- automated tests of the web application;
- load testing;
- usability testing with farmers, cooperative managers and government users.

Some security tests check that protections are declared rather than exercising them end to end. These gaps are addressed in the hardening phase of the roadmap (Chapter 10).

8 Security, privacy and data governance

8.1 Controls in place

- **Authentication.** Signed JSON Web Tokens (HMAC-SHA-256), validated for issuer, audience, signature and lifetime (120 minutes by default). Passwords are hashed with BCrypt (cost factor 11). Optional time-based two-factor authentication (TOTP, RFC 6238) is available for every account.
- **Account recovery and activation.** Password-reset and activation codes are stored only as BCrypt hashes. Reset codes expire after 10 minutes and SMS activation codes after 15 minutes; cooperative-issued activation codes are valid for seven days. Codes are cleared after use.
- **Authorisation.** Role-based access control is declared on every protected endpoint. Ownership checks restrict access to contracts, facilities and cooperative data to the parties concerned. Government users have read access to contracts and warehouse use for oversight.
- **Data access.** All database access goes through parameterised queries generated by Entity Framework Core. No user input is concatenated into SQL.
- **Input validation.** Request models are validated automatically. Shared validators cover e-mail, phone numbers, the 16-digit national identifier, coordinates, quantities, prices and date windows. Role-application documents are restricted to PDF and common image types up to 8 MB.
- **Audit.** Every API request is logged with user, role, path, status, duration, IP address and device, together with explicit before/after records for sensitive actions such as price submissions.
- **Operations.** A maintenance mode blocks non-administrators. Forecast endpoints are rate-limited (20 requests per minute for the public preview, 120 for authenticated users). The forecasting service accepts only requests carrying a service key.

8.2 Findings and remediation plan

The audit identified the issues in Table 8.1. None affects a production system, because RASS has not been deployed. All must be resolved before a pilot. They are listed by priority without operational detail.

8.3 Personal data protection

RASS processes personal data of farmers and other users: names, phone numbers, national identification numbers, farm locations, transactions and, during deliveries, transporters' locations. In Rwanda this processing falls under Law N° 058/2021 of 13 October 2021 relating to the protection of personal data and privacy (Republic of Rwanda, 2021), supervised by the National Cyber Security Authority (NCSA). Before a pilot the operating institution should:

- designate the data controller for the pilot (for example MINAGRI, RAB or a mandated operator) and complete the required registration with the supervisory authority;

Table 8.1: Security findings and planned remediation

Priority	Finding	Remediation before pilot
Critical	Development credentials and third-party test keys are present in the code repository and its history.	Rotate every key; move secrets to environment variables or a secret store; purge repository history; add secret scanning.
Critical	Demonstration accounts share one default password.	Remove demonstration seeding from pilot builds; force password change on first login; enforce a password policy.
Critical	No account lockout or rate limit on sign-in, reset and code verification.	Per-account and per-IP rate limits; lockout with back-off; attempt limits on one-time codes.
High	One-time codes are generated with a non-cryptographic generator, are written to developer logs, and appear in plain text in notification records.	Cryptographic generation; delivery by SMS only; never stored or logged in clear.
High	Uploaded documents, including identity documents in role applications, are served from a public path.	Serve through an authorised endpoint; store outside the web root; malware scanning; size and type limits on all uploads.
High	Real-time hubs do not check that a subscriber is a party to a delivery, and accept client-sent broadcast events.	Server-side authorisation on group membership; remove client broadcast methods.
High	No HTTPS enforcement, security headers or global error handler.	HTTPS with HSTS behind a reverse proxy; standard security headers; generic error responses.
High	Two-factor secrets are stored unencrypted, and the QR code is rendered by an external service.	Encrypt at rest; render QR codes locally.
Medium	Deployment configuration incomplete (container build, cross-origin settings, forecasting service not orchestrated).	Production compose or Kubernetes manifests; per-environment configuration; health checks.
Medium	Schema changes partly applied by start-up scripts rather than migrations.	Consolidate into versioned migrations; run migrations as a deployment step.
Medium	One dependency (`Microsoft.OpenApi`) has a published high-severity advisory.	Upgrade; automate dependency scanning.

- carry out a data protection impact assessment, in particular for national identifiers and location tracking;
- define retention periods for audit logs, which currently record every request, and for location history;
- ensure that data are hosted in accordance with national requirements, and document any transfer to external services such as the LLM provider used by the assistant;
- publish a privacy notice in Kinyarwanda and English and record consent where required.

8.4 Data governance for price information

Price data are the platform's most policy-relevant asset. We recommend the following:

- Market agents should be appointed and accredited by the responsible institution.

- Moderation decisions should be retained with reasons; the platform already does this.
- Official price series should be linked to the platform through a documented interface rather than re-keyed.
- The definitions of composite indicators should be agreed with MINAGRI and NISR before publication (Section 4.1).

9 Limitations and corrections

9.1 Corrections made in version 2.0

The independent code audit carried out for this report found defects in the forecasting and analytics components, as well as in some presentation layers. In the interest of transparency towards MINAGRI, RAB and future readers, they are listed in Table 9.1 together with the correction applied. Each correction is covered by an automated test.

Table 9.1: Defects found by the audit and corrections applied

Defect	Correction
The administrative "model training" screen displayed simulated progress (loss curves, processor utilisation) and recorded generated accuracy figures, not measured ones.	Replaced by a real rolling-origin evaluation on verified platform prices. Only measured errors are stored. Models worse than the naive benchmark are marked degraded.
A model-performance endpoint evaluated on synthetic series, adjusted the ensemble's figures by fixed factors, and labelled two models under the names of others.	Rewritten to serve the results of the WFP evaluation in Chapter 6, with full provenance.
The role-advice forecast reused a model fitted for the first crop and market requested for all later requests.	A new model is fitted for every request; all forecasts now come from the evaluated ensemble.
Forecasts and analytics used price submissions that moderation had flagged or rejected.	Flagged and rejected prices are excluded from all forecasting and analytics queries.
The 30-day horizon offered in the interface failed on one forecast path, which accepted at most 14 days; the ensemble view was fixed at 14 days.	All paths accept 1–30 days; the ensemble view follows the selected horizon.
The reported "ensemble accuracy" measured the smoothness of the forecast path, not its accuracy.	Defined as one minus the hold-out error of the ensemble and reported with that error.
Season boundaries differed between modules.	Season A (September–February), B (March–June) and C (July–August) used throughout.
A listing-rejection message referred to the wrong regulatory authority.	Message refers to the government price regulation in force.

Figures shown in earlier demonstrations of the platform's administrative dashboard, notably model accuracy percentages, were produced by the simulated component. They should not be cited. The national price intelligence and district-level price regulation demonstrated at AgriShow 2026 were functional, as described in Chapter 4. The 30-day ensemble forecast was available through the platform's API. Its interface was limited to 14 days and its accuracy had not then been measured.

9.2 Remaining limitations

1. **Demonstration data.** The price history in the demonstration database is synthetic: 8 crops in 8 markets over 61 days, generated for software testing. Dashboards built on it illustrate functionality only. Real-data accuracy is established solely by the WFP evaluation.
2. **Evaluation scope.** The evaluation is monthly, covers six crops in ten markets of three

provinces, and includes no Kigali or Northern Province series. Daily-horizon accuracy has not yet been measured on real data.

3. **Exogenous drivers.** Forecasts use price history only. Rainfall, input and fuel prices, cross-border trade and policy events are not yet model inputs.
4. **Payments and messaging.** Card payments run in test mode. Mobile money, SMS and e-mail are not connected. Escrow and farmer payouts are ledger records, not transfers (Table 3.3).
5. **Workflow gaps.** Several workflows are not yet connected:
   - approving a harvest declaration does not create inventory;
   - accepted crop-sharing requests do not generate contracts or transport;
   - role approval does not create the corresponding profile;
   - transport pricing is a flat placeholder.
6. **Indicators.** The food-security index and regional scores are heuristic and unvalidated.
7. **Reference data.** Sector coverage is 402 of 416, and cells and villages are not held.
8. **Language.** About 5% of interface strings are not yet translated into Kinyarwanda, and the runtime translation layer can mis-assign strings when the catalogues diverge.
9. **Field validation.** No usability study or field pilot has been carried out.

10 Pilot roadmap and recommendations

10.1 Proposed phases

Table 10.1: Proposed path from prototype to supervised pilot

Phase	Duration	Main deliverables
0. Hardening	3 months	Resolve all findings in Table 8.1; production deployment configuration with backups and monitoring; data protection impact assessment and registration; end-to-end and interface tests; complete Kinyarwanda translation; connect SMS for codes and price alerts; replace demonstration data with verified historical prices.
1. Supervised pilot	6 months	Two to three districts selected with MINAGRI/RAB, ideally covering one urban and two rural markets; 10–20 accredited market agents; participating cooperatives and buyers; government users exercising price regulation and monitoring; monthly re-evaluation of forecasts with the in-platform tool and publication of the results; usability study with farmers and cooperative managers.
2. Evaluation and decision	2 months	Independent review of pilot results against the indicators in Table 10.2; decision on scale-up, scope and institutional ownership.
3. Scale-up (conditional)	12+ months	Additional districts; mobile-money disbursement; offline-capable mobile client; integration of official price series and weather data; exogenous-variable forecasting models evaluated with the same protocol.

10.2 Pilot success indicators

Table 10.2: Proposed indicators for the supervised pilot

Area	Indicator and target to be agreed with MINAGRI/RAB
Price data	Share of pilot markets with weekly verified prices for priority crops; median delay between observation and publication.
Forecast quality	Out-of-sample MAPE and skill against naive on pilot data at 1–3 months, re-estimated monthly; interval coverage close to nominal.
Regulation	Share of listings checked against an active regulation; number of violations detected and resolved; time from regulation creation to notification of affected cooperatives.
Trade	Number of transactions completed end to end; share delivered with tracking; farmer payment allocations reconciled with cooperative records.
Adoption	Active users by role; farmer and manager satisfaction from the usability study.
Security	No unresolved critical or high findings; successful restore test of backups.

10.3 Recommendations

To MINAGRI and RAB.

1. Nominate counterparts for price regulation and market monitoring to co-own the pilot and validate indicator definitions.
2. Facilitate access to official historical and ongoing market price series so that forecasts are trained and evaluated on the data used for policy.
3. Adopt out-of-sample evaluation against a naive benchmark, with published metrics, as the acceptance criterion for forecasting tools used in public decision support.
4. Treat composite food-security indicators in RASS as indicative until validated jointly with NISR.

To the development team.

1. Complete Phase 0 before any pilot data are collected, beginning with credential rotation and authentication protections.
2. Extend the forecasting evaluation to daily data as soon as verified pilot data allow. Test exogenous drivers (rainfall, fuel prices, seasonal calendars) and direct multi-horizon models across series, retaining a model only if it improves on the statistical ensemble out of sample.
3. Connect the remaining workflow steps (Chapter 9) and replace the placeholder transport tariff with a distance-based one.
4. Publish the evaluation code and results with each model change, as done for this report.

11 Conclusion

RASS brings market price collection, government price regulation, price forecasting and trade coordination into one platform for Rwanda's agricultural value chain. The platform is extensive and largely functional. Price regulation is enforced where it matters, at the point of listing, and is monitored continuously against observed prices. Users of every role can see current prices across markets, the trade cycle is digitised from lot to delivery, and the government dashboard gathers the signals that matter for market supervision.

For the first time, the forecasting component has been evaluated rigorously on real Rwandan retail prices. The evaluation changed the system:

- it exposed two machine-learning components that performed far worse than a naive forecast;
- it showed that a weighted ensemble of statistical models reduces the naive error by 15.4–22.1% two to three months ahead;
- it showed that one month ahead, no model beats the latest observed price.

The production system now uses the evaluated ensemble and calibrated intervals. The audit also led to the removal of simulated performance figures and the correction of several defects, which are documented openly in this report.

RASS is ready for a supervised pilot once a defined set of hardening tasks is completed. The most valuable next step is not more functionality but better data: verified prices collected through the platform under MINAGRI and RAB supervision. With such data, every claim the platform makes can be tested in the same transparent way as in this report.

References

Bates, J. M., & Granger, C. W. J. (1969). The combination of forecasts. *Operational Research Quarterly*, 20(4), 451–468.

Box, G. E. P., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). *Time Series Analysis: Forecasting and Control* (5th ed.). Wiley.

Diebold, F. X., & Mariano, R. S. (1995). Comparing predictive accuracy. *Journal of Business & Economic Statistics*, 13(3), 253–263.

Elman, J. L. (1990). Finding structure in time. *Cognitive Science*, 14(2), 179–211.

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189–1232.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.

Holt, C. C. (2004). Forecasting seasonals and trends by exponentially weighted moving averages. *International Journal of Forecasting*, 20(1), 5–10. (Reprint of ONR Memorandum 52, 1957.)

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts. https://otexts.com/fpp3/

Hyndman, R. J., & Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4), 679–688.

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2018). Statistical and machine learning forecasting methods: Concerns and ways forward. *PLoS ONE*, 13(3), e0194889.

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2020). The M4 Competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting*, 36(1), 54–74.

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Republic of Rwanda (2021). Law N° 058/2021 of 13/10/2021 relating to the protection of personal data and privacy. *Official Gazette of the Republic of Rwanda*, Special issue of 15/10/2021.

Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and statistical modeling with Python. In *Proceedings of the 9th Python in Science Conference* (pp. 92–96).

Tashman, L. J. (2000). Out-of-sample tests of forecasting accuracy: An analysis and review. *International Journal of Forecasting*, 16(4), 437–450.

Taylor, S. J., & Letham, B. (2018). Forecasting at scale. *The American Statistician*, 72(1), 37–45.

World Food Programme (2026). *Rwanda – Food Prices* [dataset]. WFP Vulnerability Analysis and Mapping, distributed by the Humanitarian Data Exchange. Licence CC BY-IGO. https://data.humdata.org/dataset/wfp-food-prices-for-rwanda (accessed 1 November 2026).

A Reproducibility

A.1 Software environment

Python	3.13; pandas 3.1.0, NumPy 2.5.4, SciPy 1.18.1, statsmodels 0.15.0, Prophet 1.5.0, scikit-learn 1.9.1, FastAPI 0.162.0
.NET	SDK 10.0.302 (target `net10.0`)
Data file	`forecasting-service/data/wfp_food_prices_rwa.csv`, 153,204 records, SHA-256 `3880fb6d2a11aec08a6e34e3caa1494c8d5003d2c35b42933e8416bba3b296e2`
Runtime	Full evaluation about two minutes on a standard laptop (seven worker processes)

A.2 Commands

From the repository root:

```
# forecasting environment
cd forecasting-service
python3.13 -m venv .venv
.venv/bin/pip install -r requirements.txt

# evaluation (writes data/backtest_results.json and data/backtest_errors.csv)
.venv/bin/python evaluation/backtest_wfp.py

# report tables and figures
.venv/bin/python ../report/tables/make_tables.py
.venv/bin/python ../report/figures/make_figures.py

# test suites
.venv/bin/python -m unittest discover -s tests
cd ../backend/testing/Rass.Api.Tests && dotnet test

# report
cd ../../../report && tectonic main.tex
```

A.3 Series used in the evaluation

Table A.1: The 28 WFP retail price series retained for the evaluation

Crop	Market	Province
Beans	Mahama (Camp)	Eastern
Beans	Mugera	Eastern
Beans	Nyabiheke (Camp)	Eastern
Beans	Gasarenda	Southern

Crop	Market	Province
Beans	Kigeme	Southern
Beans	Kigeme (Camp)	Southern
Beans	Mugombwa (Camp)	Southern
Beans	Karongi	Western
Beans	Kiziba (Camp)	Western
Beans	Mubuga	Western
Cassava	Mahama (Camp)	Eastern
Maize	Mahama (Camp)	Eastern
Maize	Mugera	Eastern
Maize	Gasarenda	Southern
Maize	Kigeme	Southern
Maize	Mugombwa (Camp)	Southern
Maize	Karongi	Western
Potatoes	Mahama (Camp)	Eastern
Potatoes	Mugera	Eastern
Potatoes	Gasarenda	Southern
Potatoes	Kigeme	Southern
Potatoes	Kigeme (Camp)	Southern
Potatoes	Mugombwa (Camp)	Southern
Rice	Mugera	Eastern
Sorghum	Gasarenda	Southern
Sorghum	Kigeme	Southern
Sorghum	Kigeme (Camp)	Southern
Sorghum	Mugombwa (Camp)	Southern

B API endpoints by controller

Table B.1: Number of HTTP endpoints per backend controller (total 437)

Controller	Endpoints
Admin	55
Cooperative	51
AIInsights	36
Storage	24
Government	24
Contracts	18
Warehouse	17
Farmers	17
Reference	13
Transporters	12
TransportJobs	12
Profile	12
MarketAgent	12
Forecast	11
CropSharing	11
Catalog	10
Buyers	10
Chat	9
Applications	9
PriceRegulation	8
Payments	8
Notifications	8
Auth	8
Tracking	7
Lots	6
Cart	6
MarketPrices	5
MarketListings	5
BuyerOrders	5
Transport	2
RoleAnalytics	2
Reports	2
Platform	1
AIChat	1

C Glossary

Naive forecast
A forecast equal to the most recent observed value; the minimum benchmark for any forecasting model.

Rolling-origin evaluation
Repeatedly fitting a model on data up to a cut-off date and scoring its forecasts for the following periods, moving the cut-off forward each time.

Skill
Percentage reduction in forecast error relative to the naive forecast; positive values indicate an improvement.

Coverage
Share of observed values that fall inside a forecast interval.

Inverse-RMSE weighting
Combining forecasts with weights proportional to the inverse of each model's recent root mean squared error.

Price regulation
In RASS, a government-defined minimum and maximum price per kilogram for a crop in a province, district or market over a period.

Moderation
Classification of a submitted market price as approved, pending, flagged or rejected.